Relatório da Aplicação Biblioteca de Jogos

Programação Dispositivos Móveis [Multimédia (ISLA GAIA)]



Trabalho Final de Programação De Dispositivos Móveis

Pedro Sá Oliveira / a22406977

Gonçalo Guimarães / a22407056

Professor Arnaldo

1. Introdução 3
2. Funcionalidades da Aplicação 3
3. Descrição da Interface 4
3.1 Ecrã Principal (MainActivity) 4
3.2 Ecrã Adicionar Jogo (AddGameActivity) 6
3.3 Ecrã Editar Jogo (EditGameActivity) 7
4. Estrutura da Aplicação 8
5. Blocos de Código Mais Importantes 9
5.1 Classe Game 9
5.2 DAO (GameDao) 10
5.3 Base de Dados Room 12
5.4 Repository 13
5.5 ViewModel 14
5.6 MainActivity 15
5.7 AddGameActivity 18
5.8 EditGameActivity 19
5.9 GameAdapter 21
6. Arquitetura Utilizada 23
7. Conclusão 24

1. Introdução

A aplicação Biblioteca de Jogos tem como objetivo permitir ao utilizador gerir uma coleção pessoal de videojogos. A app possibilita adicionar,editar, visualizar, pesquisar e eliminar jogos armazenados localmente no dispositivo.

O projeto foi desenvolvido em Android Studio utilizando Kotlin e segue a arquitetura MVVM (Model-View-ViewModel), promovendo uma separação clara entre a interface gráfica, a lógica de negócio e o acesso aos dados.

2. Funcionalidades da Aplicação

A aplicação disponibiliza as seguintes funcionalidades:

Gestão de Jogos

- Adicionar novos jogos.
- Edição do conteúdo de um jogo.
- Eliminar jogos da biblioteca.
- Visualizar todos os jogos guardados.

Pesquisa

- Pesquisa em tempo real por:
    - Nome do jogo;
    - Plataforma;
    - Género.

Gestão de Imagens

- Seleção de uma imagem da galeria do dispositivo.
- Associação da imagem ao jogo.

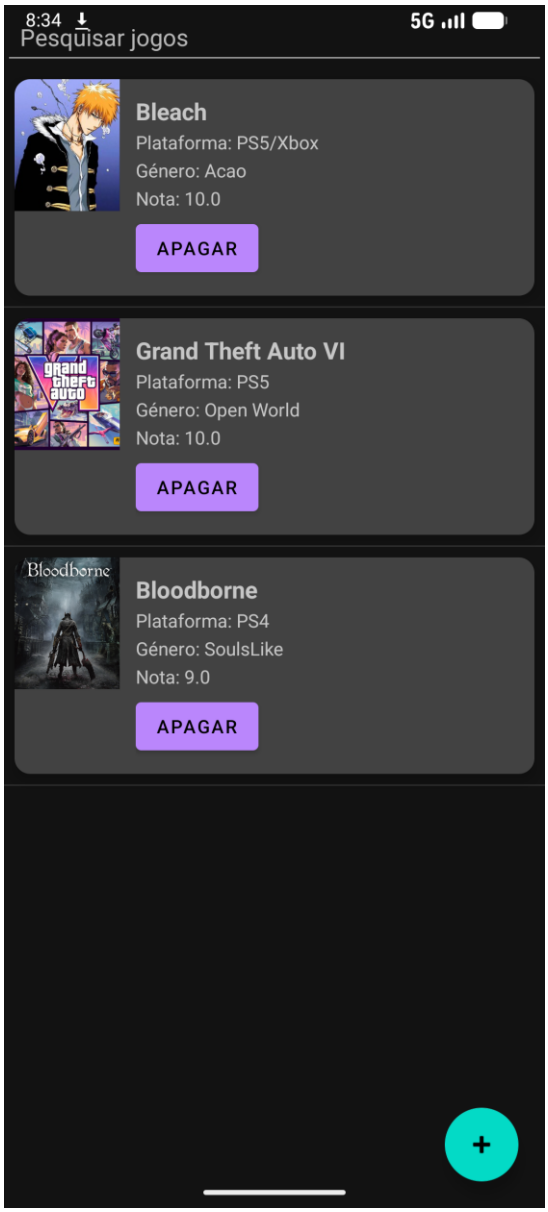
Ordenação

- Os jogos são apresentados por ordem decrescente de classificação (rating).

3. Descrição da Interface

3.1 Ecrã Principal (MainActivity)

Este é o ecrã inicial da aplicação.



Componentes

Campo de Pesquisa

Permite filtrar os jogos existentes através de texto introduzido pelo utilizador.

RecyclerView

Apresenta a lista de jogos armazenados na base de dados.

Cada item mostra:

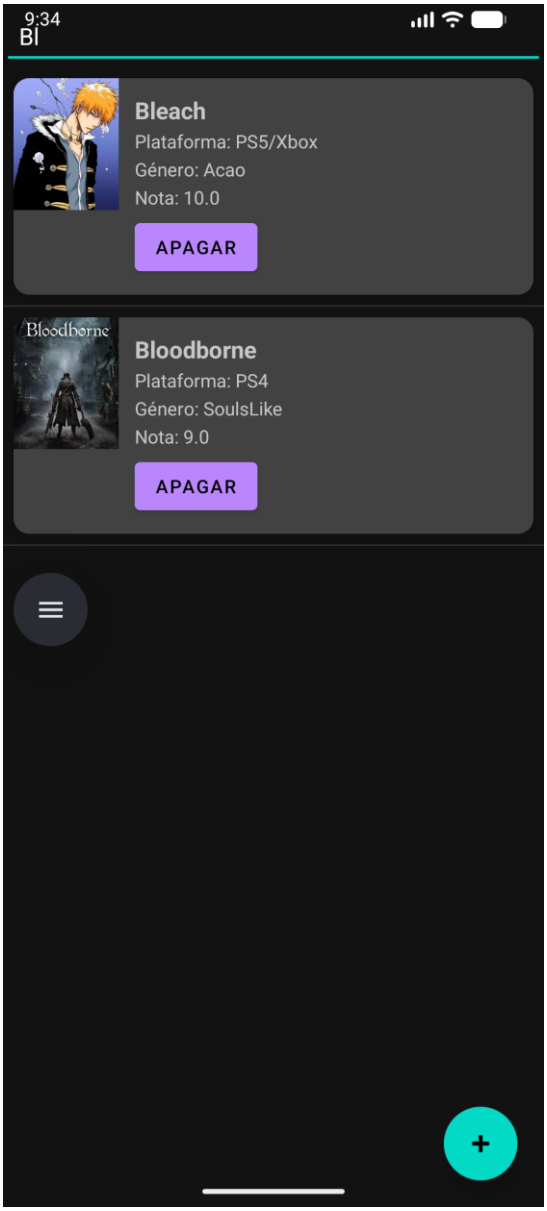
- Imagem do jogo;
- Nome;
- Plataforma;
- Género;
- Classificação;
- Botão de eliminação.

Floating Action Button (+)

Permite abrir o formulário de adição de um novo jogo.

Comportamento

- Os dados são atualizados automaticamente através de LiveData.
- A pesquisa é realizada à medida que o utilizador escreve.



3.2 Ecrã Adicionar Jogo (AddGameActivity)

Permite inserir um novo jogo na biblioteca.

Campos disponíveis

- Nome do jogo
- Plataforma
- Género
- Nota/Classificação
- Imagem de capa

Botões

Escolher capa

Abre a galeria do dispositivo para seleção de uma imagem.

Guardar

Cria um novo objeto Game e guarda-o na base de dados.

Validação

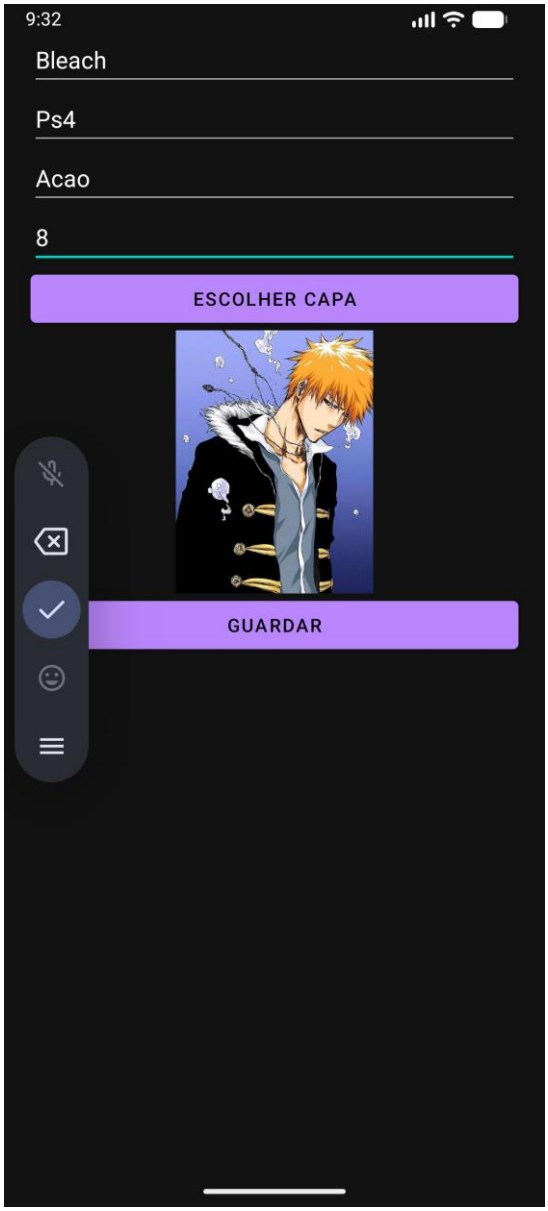
O nome do jogo é obrigatório.

3.3 Ecrã Editar Jogo (EditGameActivity)

Permite alterar os dados de um jogo existente.

Funcionalidades

- Carrega os dados atuais do jogo.
- Permite alterar:
    - Nome;
    - Plataforma;
    - Género;
    - Nota.
- Guarda as alterações na base de dados.



4. Estrutura da Aplicação

O projeto está organizado em vários módulos:

Database

Contém:

- Game.kt
- GameDao.kt
- GameDatabase.kt

Repository

Contém:

- GameRepository.kt

ViewModel

Contém:

- GameViewModel.kt
- GameViewModelFactory.kt

Activities

Contém:

- MainActivity.kt
- AddGameActivity.kt
- EditGameActivity.kt

Adapter

Contém:

- GameAdapter.kt

5. Blocos de Código Mais Importantes

5.1 Classe Game

Ficheiro:

Game.kt

Função

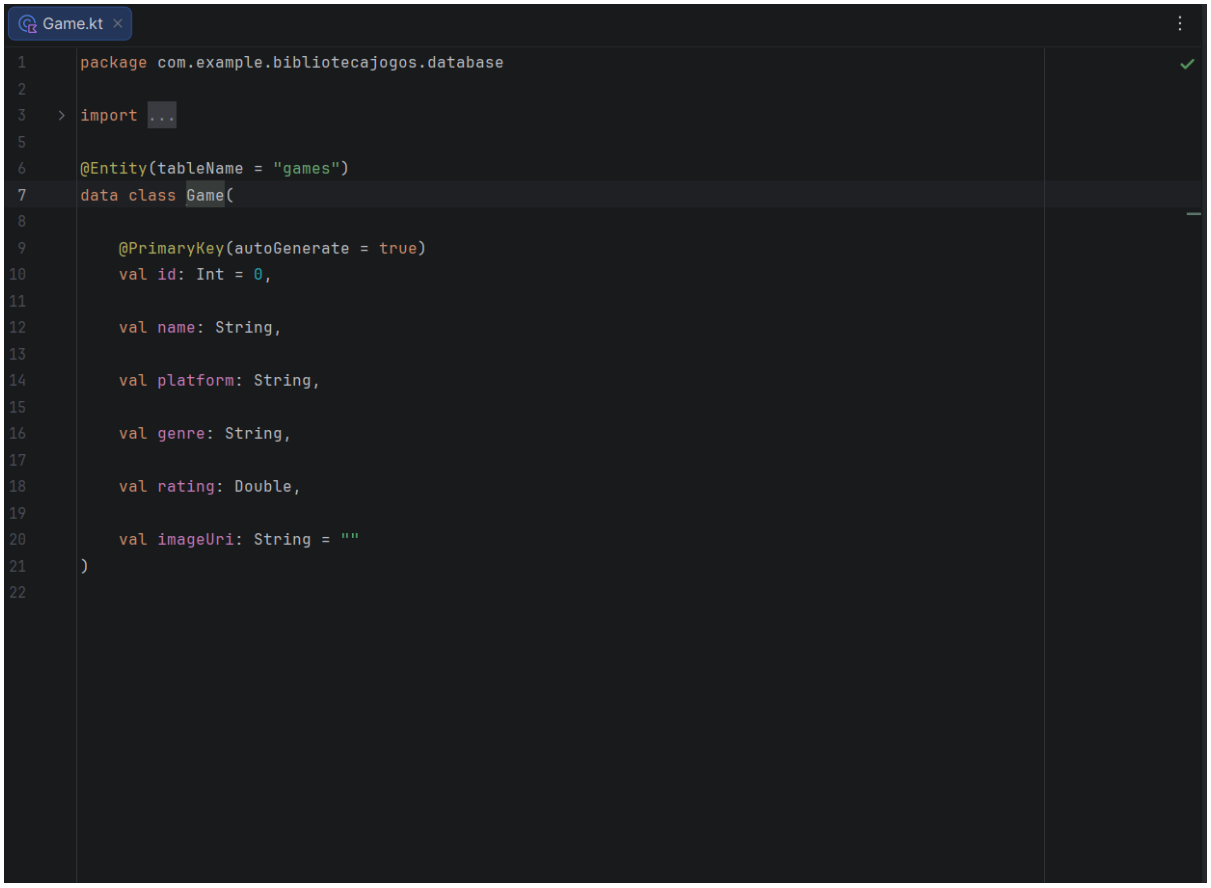
Representa a entidade da base de dados.

Campos:

- id
- name
- platform
- genre
- rating
- imageUri

Importância

É o modelo principal utilizado em toda a aplicação.



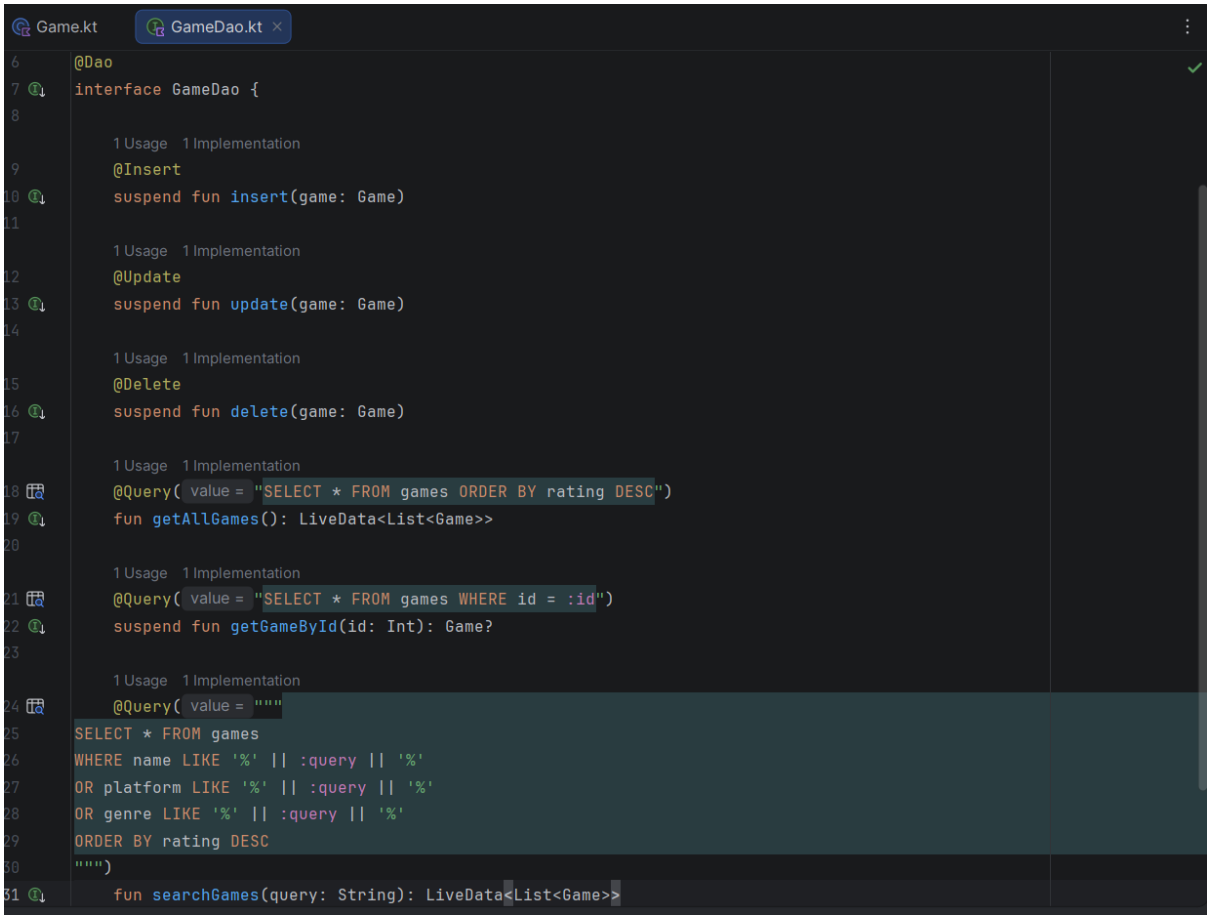
5.2 DAO (GameDao)

Ficheiro:

GameDao.kt

Função

Define todas as operações de acesso à base de dados.



Métodos:

Inserção

insert(game)

Adiciona um jogo.

Atualização

update(game)

Atualiza um jogo existente.

Eliminação

delete(game)

Remove um jogo.

Obter todos os jogos

getAllGames()

Retorna todos os jogos ordenados por classificação.

Pesquisa

searchGames(query)

Executa uma pesquisa por:

- nome;
- plataforma;
- género.

Importância

Centraliza todas as consultas SQL da aplicação.

5.3 Base de Dados Room

Ficheiro:

GameDatabase.kt

Função

Cria e gere a base de dados local SQLite através da biblioteca Room.

```kotlin
package com.example.bibliotecajogos.database

import ...

6 Usages  1 Implementation
@Database(
    entities = [Game::class],
    version = 1
)
abstract class GameDatabase : RoomDatabase() {

    1 Usage  1 Implementation
    abstract fun gameDao(): GameDao

    4 Usages
    companion object {

        2 Usages
        @Volatile
        private var INSTANCE: GameDatabase? = null

        1 Usage
        fun getDatabase(context: Context): GameDatabase {

            return INSTANCE ?: synchronized( lock = this) {

                val instance =
                    Room.databaseBuilder(
                        context.applicationContext,
                        klass = GameDatabase::class.java,
                        name = "games_database"
                    )
```

Características

- Singleton (apenas uma instância).
- Tabela principal: games.
- Nome da base de dados:

games_database

Importância

Responsável pelo armazenamento persistente dos dados.

5.4 Repository

Ficheiro:

GameRepository.kt



Função

Atua como camada intermédia entre a ViewModel e a base de dados.

Responsabilidades

- Encapsular o acesso ao DAO.
- Fornecer métodos simples para:

- inserir;
- atualizar;
- eliminar;
- pesquisar.

Importância

Reduz o acoplamento entre a interface e a base de dados.

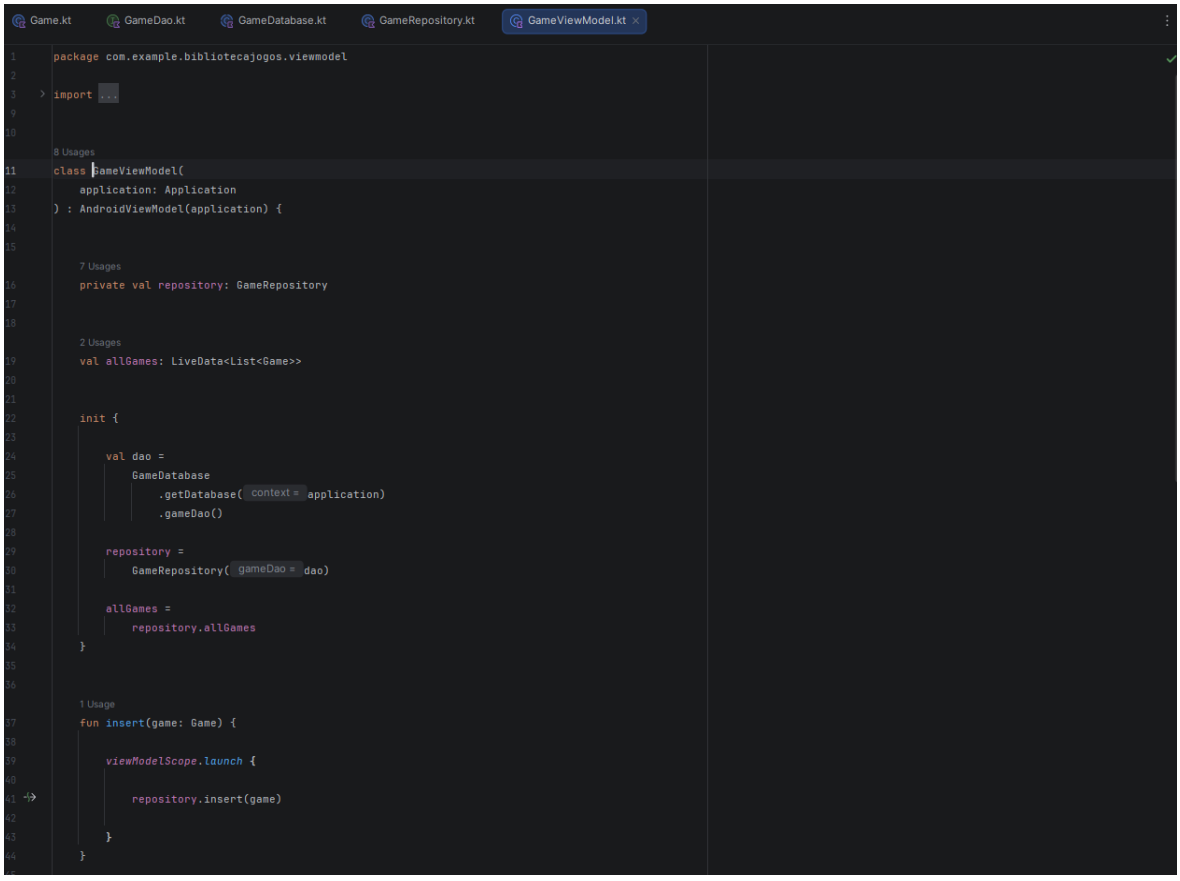
5.5 ViewModel

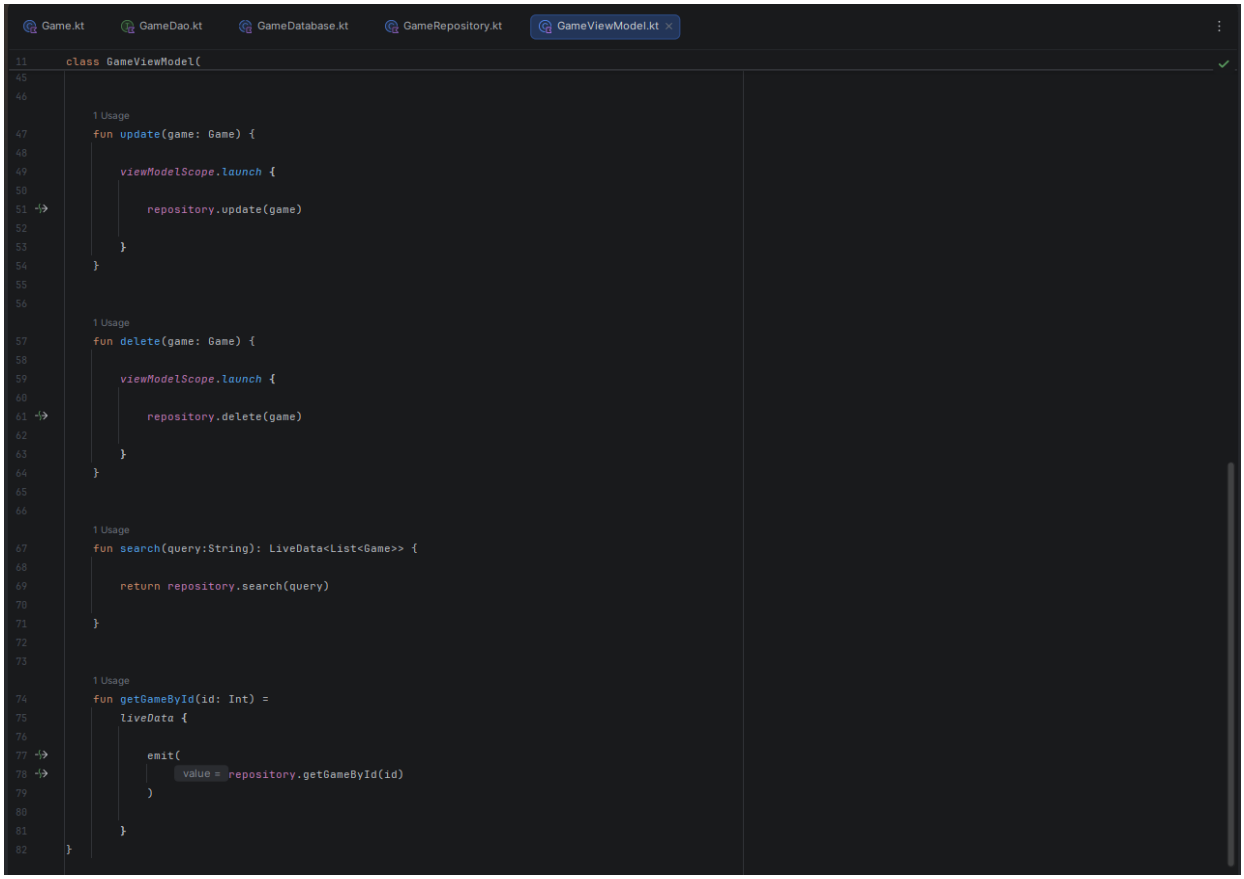
Ficheiro:

GameViewModel.kt

Função

Gerir os dados apresentados na interface.





Características

Utiliza:

- LiveData
- Coroutines
- ViewModelScope

Operações

- insert()
- update()
- delete()
- search()
- getGameById()

Importância

Mantém os dados mesmo quando ocorrem alterações de configuração do dispositivo.

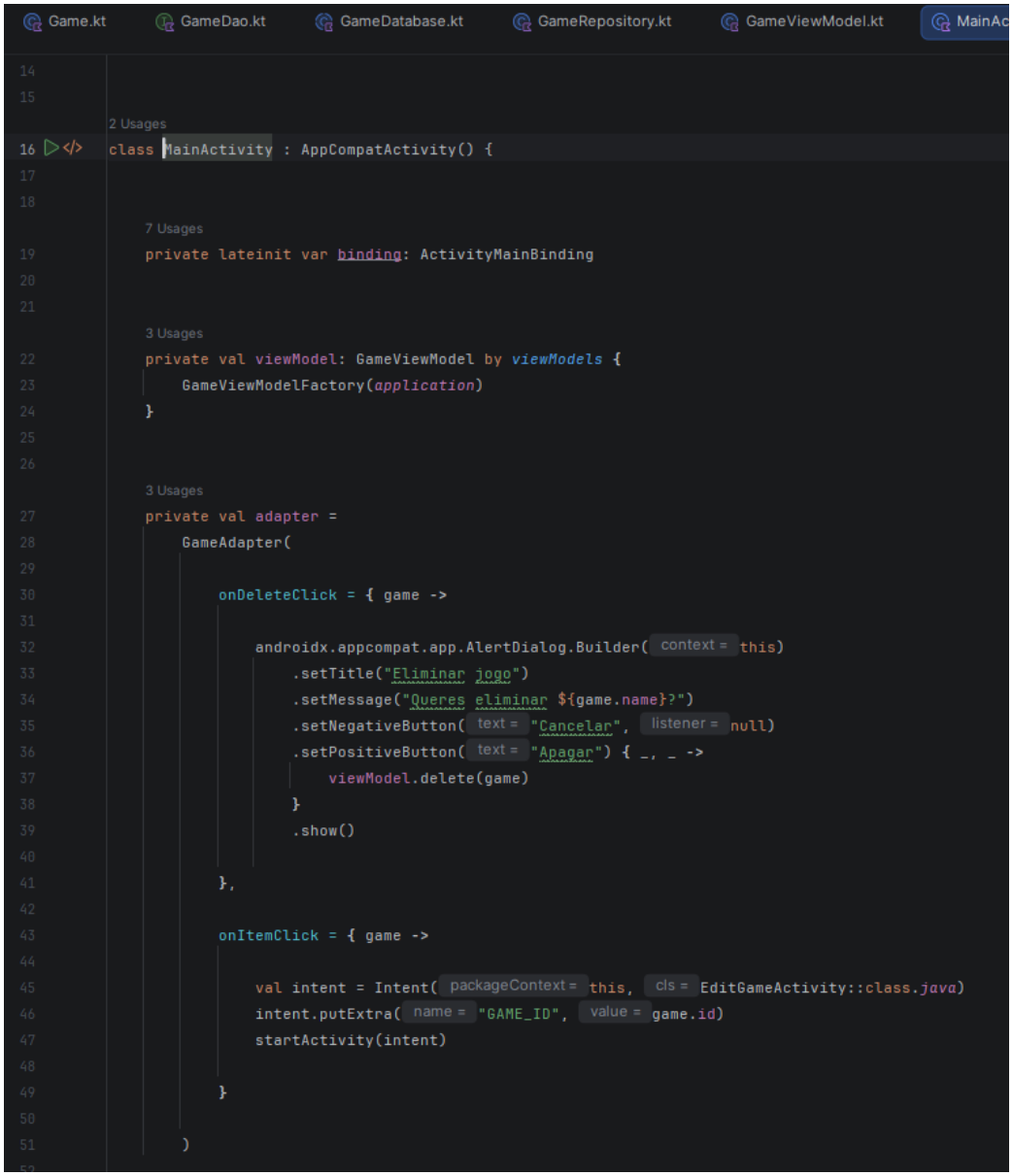
5.6 MainActivity

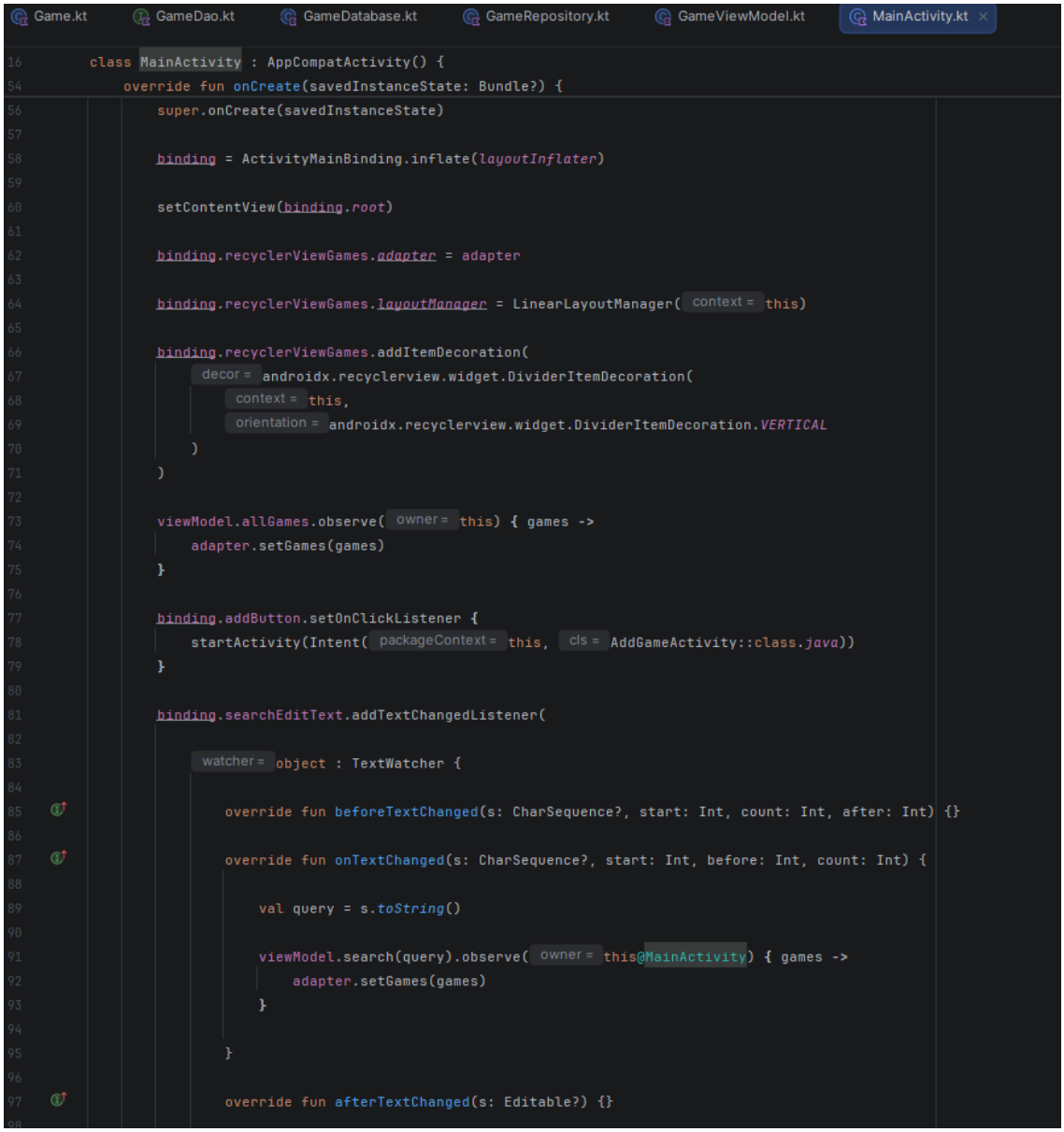
Ficheiro:

MainActivity.kt

Função

Controla o ecrã principal.





Funcionalidades implementadas

Observação de dados

Observa allGames e atualiza automaticamente a RecyclerView.

Pesquisa dinâmica

Utiliza TextWatcher para pesquisar à medida que o utilizador escreve.

Eliminação

Apresenta uma caixa de confirmação antes de apagar um jogo.

Edição

Ao selecionar um item, abre o EditGameActivity.

Importância

É o centro de interação principal da aplicação.

5.7 AddGameActivity

Ficheiro:

AddGameActivity.kt

Função

Criar novos jogos.





Destaques

Seleção de imagem

Utiliza:

ActivityResultContracts.GetContent()

para abrir a galeria do dispositivo.

Criação do objeto

Cria um objeto Game com os dados introduzidos.

Inserção

Envia o jogo para a ViewModel, que posteriormente o guarda na base de dados.

5.8 EditGameActivity

Ficheiro:

EditGameActivity.kt

Função

Editar jogos existentes.





```
        binding.saveButton.setOnClickListener {

            val updatedGame = Game(
                id = gameId,
                name = binding.nameInput.text.toString(),
                platform = binding.platformInput.text.toString(),
                genre = binding.genreInput.text.toString(),
                rating = binding.ratingInput.text.toString().toDoubleOrNull() ?: 0.0,
                imageUri = currentGame?.imageUri ?: ""
            )

            viewModel.update(updatedGame)

            finish()

        }

    }

}
```

Processo

1. Recebe o ID do jogo.
2. Carrega os dados da base de dados.
3. Preenche os campos do formulário.
4. Guarda as alterações efetuadas.

Importância

Permite manter os dados atualizados sem criar novos registos.

5.9 GameAdapter

Ficheiro:

GameAdapter.kt

Função

Ligação entre a RecyclerView e os dados.

```
3 Usages
class GameAdapter(
    private val onDeleteClick: (Game) -> Unit,
    private val onItemClick: (Game) -> Unit
) :
    RecyclerView.Adapter<GameAdapter.GameViewHolder>() {


    3 Usages
    private var games = emptyList<Game>()


    4 Usages
    class GameViewHolder(
        itemView: View
    ) :
        RecyclerView.ViewHolder(itemView) {

        2 Usages
        val image: ImageView = itemView.findViewById(R.id.gameImage)
        val name: TextView = itemView.findViewById(R.id.gameName)
        1 Usage
        val platform: TextView = itemView.findViewById(R.id.gamePlatform)
        1 Usage
        val genre: TextView = itemView.findViewById(R.id.gameGenre)
        1 Usage
        val rating: TextView = itemView.findViewById(R.id.gameRating)
        1 Usage
        val deleteButton: Button = itemView.findViewById(R.id.deleteButton)

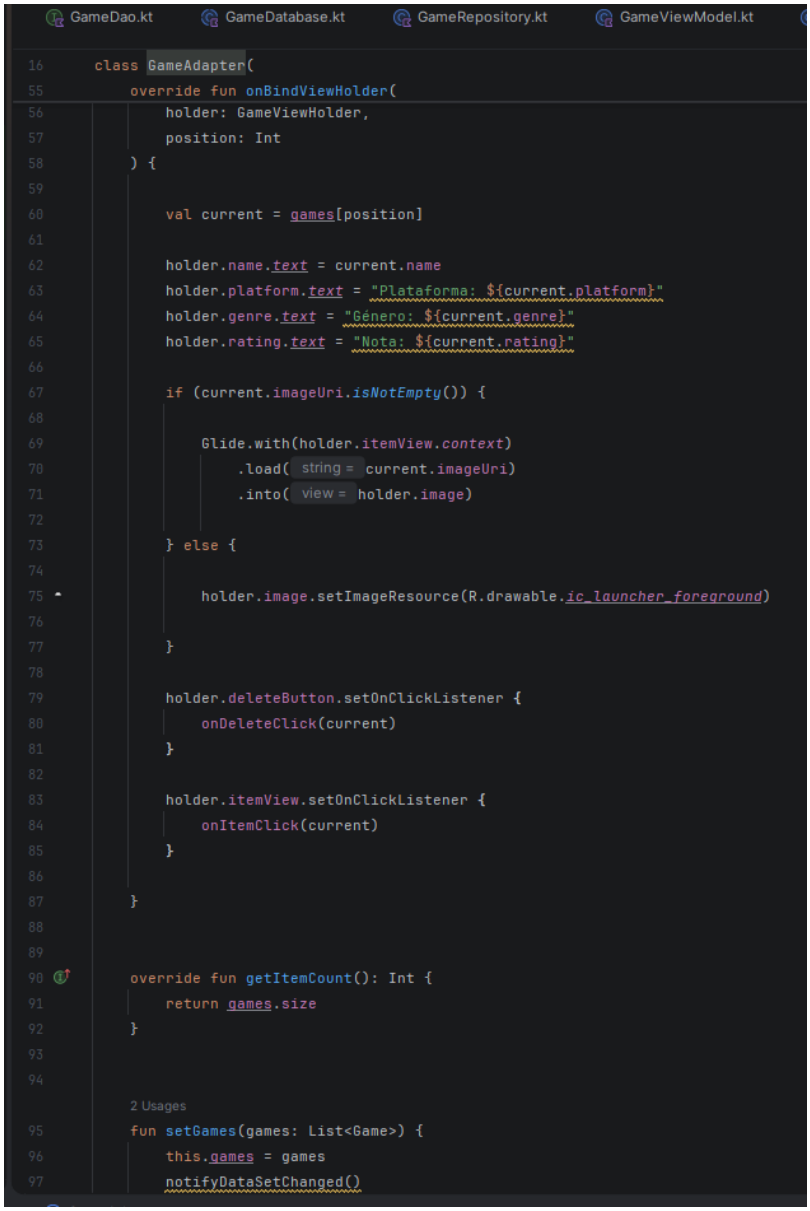
    }


    override fun onCreateViewHolder(
        parent: ViewGroup,
        viewType: Int
    ): GameViewHolder {

        val view =
            LayoutInflater.from(parent.context)
                .inflate( resource = R.layout.item_game, root = parent, attachToRoot = false)

        return GameViewHolder( itemView = view)

    }
```



Responsabilidades

- Criar cada item visual da lista.
- Preencher os dados dos jogos.
- Gerir cliques.
- Gerir eliminações.

Biblioteca utilizada

Glide

Responsável por carregar imagens de forma eficiente.

Importância

É o componente responsável pela apresentação visual da coleção de jogos.

6. Arquitetura Utilizada

A aplicação segue o padrão MVVM.

Fluxo:

Interface (Activity)

↓

ViewModel

↓

Repository

↓

DAO

↓

Room Database

Vantagens

- Código mais organizado.
- Maior facilidade de manutenção.
- Separação clara de responsabilidades.
- Atualização automática da interface através de LiveData.

7. Conclusão

A aplicação Biblioteca de Jogos constitui um sistema completo de gestão de videojogos armazenados localmente. O projeto demonstra a utilização prática de tecnologias Android modernas, incluindo Room, LiveData, ViewModel, RecyclerView, Glide e Coroutines.

As funcionalidades implementadas permitem ao utilizador criar, consultar, pesquisar, editar e eliminar jogos de forma intuitiva, enquanto a arquitetura MVVM garante uma estrutura organizada, escalável e de fácil manutenção.